Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Фізико-технічний інститут

КРИПТОГРАФІЯ

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №4

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Варіант 1

Виконали:
студенти гр. ФБ-41 Пінчук А.П.,
Плотко В.О.

Перевірила:
Селюх П.В.

Київ 2026

Мета роботи: Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація засекреченого зв'язку, електронного підпису та протоколу конфіденційного розсилання ключів.

Варіант 1 (спільне завдання з індивідуальною генерацією випадкових простих чисел довжиною щонайменше 256 біт).

Порядок виконання роботи

Постановка задачі:

- Власноруч реалізувати алгоритм швидкого піднесення до степеня за модулем (схема Горнера), розширений алгоритм Евкліда, тест пробних ділень та імовірнісний тест Міллера–Рабіна без використання сторонніх бібліотек.
- Згенерувати дві пари простих чисел p, q (для Абонента А) та p_1, q_1 (для Абонента В) довжиною 256 біт за умови $p \cdot q \le p_1 \cdot q_1$.
- Реалізувати 7 окремих високорівневих процедур, кожна з яких приймає на вхід лише необхідні ключові дані: **GenerateKeyPair()**, **Encrypt()**, **Decrypt()**, **Sign()**, **Verify()**, **SendKey()**, **ReceiveKey()**.
- Перевірити роботу всіх операцій локально та у взаємодії з тестовим сервером http://asymcryptwebservice.appspot.com/?section=rsa.

Короткий опис реалізованих програмних засобів, труднощів та шляхів їх розв'язання:

Ми написали програму мовою Python у файлі **main.py**. Для генерації криптографічно стійких випадкових чисел використали вбудований модуль **secrets**. Власноруч реалізували математичну базу: алгоритм Евкліда (**gcd**), розширений алгоритм Евкліда (**extended_gcd**), пошук оберненого елемента за модулем (**mod_inverse**) та схему Горнера для швидкого піднесення до степеня (**mod_pow**). Для пошуку простих чисел написали попередній фільтр пробних ділень (**trial_division** на малі прості числа від 2 до 97) та імовірнісний тест Міллера–Рабіна (**miller_rabin** на 20 раундів). Програма автоматично зберігає всі результати обчислень у файли **results.txt**, **candidates.txt**, **keys.json** та **server_hex.txt**.

Під час виконання роботи виникли дві основні труднощі:

- **Дотримання умови протоколу $n \le n_1$:** оскільки прості числа генеруються випадково з одного діапазону 256 біт, модуль абонента А іноді виходив більшим за модуль абонента В. Ми вирішили це додаванням перевірки після генерації: якщо $n > n_1$, програма автоматично міняє ключові пари місцями, щоб завжди виконувалась нерівність $n \le n_1$.

- **Сумісність із тестовим вебсервером:** сайт приймає числа виключно у шістнадцятковому форматі (HEX) і рахує цифровий підпис за прямою формулою $S = M^d \bmod n$ без попереднього гешування SHA-256. Щоб результати повністю збігалися з сервером, ми реалізували підпис за формулою з методички і додали автоматичний запис усіх чисел у HEX-форматі у файл **server_hex.txt**.

Значення вибраних чисел p, q, p_1, q_1, відхилені кандидати та параметри RSA (Завдання 1–3)

Відхилені кандидати, що не пройшли тест простоти (з файлу candidates.txt):

Усього під час генерації чотирьох простих чисел наша програма відсіяла **216 складених чисел-кандидатів** (для p: 35, для q: 4, для p_1: 7, для q_1: 170). Приклади відхилених чисел:

Для p (Абонент A):
• `9730912845928924835926781175520929926577690376530132053713580658566739532959 1`
(відхилено тестом пробних ділень — ділиться на 7);
• `9404771495553982358478083171906119143331857341358680679219378778158850005465 1` (не пройшло тест Міллера–Рабіна).

Для q (Абонент A):
• `1092752966616641266752545037149322078787172749046771872290221548410329065816 99`
(відхилено тестом пробних ділень — ділиться на 29);
• `6941103889564704256621253059627899483644477899440622189965770979456076899670 1` (не пройшло тест Міллера–Рабіна).

Для p_1 (Абонент B):
• `1063134824671602240288572371259579087805228193525187095170393527307884900164 97`
(відхилено тестом пробних ділень — ділиться на 11);
• `1124074262518847728948901511252485369912156050430853961261296923397581095464 77` (не пройшло тест Міллера–Рабіна).

Для q_1 (Абонент B):
• `1105324129376046305990466385768434095096751797334247296786932257597079911374 35`
(відхилено тестом пробних ділень — ділиться на 5);
• `9653498445003256572814362560719322275752911163560062556168053195065131755354 9` (не пройшло тест Міллера–Рабіна).

Параметри криптосистеми RSA для Абонента A:

• **p (256 біт)** = `6759630772321889807013587884199479275454287191912162243971107783357021305 6471`

• **q (256 біт)** = `9529791023445388870473595753707036398776486077336098742340298330684883229 0163`

• **n = p · q (511 біт)** = 644178686558783666637762984656751230827449183410249449692805729993888521553745341984577394223603697090141909018983078132992702759836459678451012297679477 3
(у HEX: 7AFECC5DD18C8AA8B6229A997A85F111C4C09ABF19922EDDE39EFBFA919F6CF9BFB6ECF6461511C547A51A65A03411728CB44EE62A5D1CF3907E9AD1710AD895)

• **φ(n) = (p − 1)(q − 1)** = 644178686558783666637762984656751230827449183410249449692805729993888521553729052562781626944926209906504002503308847359723454498850148272336970393144 8140

• **Відкрита експонента e** = 65537 (у HEX: 10001)

• **Секретний ключ d** = 253997305939629960626614481689963052257688242496669912103266748077758720806108189988477349007184311002852891171216319401599307073939708584547044447094 8493
(у HEX: 307F2133E8F9427F2D79CD7DB0DAF967E92D8C96C74D659F662DFCEB1ADA42CC5533F3BD72A6AF7497D966FF183826565F0A7491AA4BCC426CF8CBDC0C1C528D)

Параметри криптосистеми RSA для Абонента B:

• **p_1 (256 біт)** = 109021390804963022018894722192438746861475690088145466360333152889193189675227

• **q_1 (256 біт)** = 74827113711622902609441731774078523257763470549259638155845588278028526046849

• **$n_1 = p_1 \cdot q_1$ (512 біт)** = 815775600676224758057541045696824762404870371950626444276565885799197011362509130829347372917449385724187022230609880936626939858755258282359326849670 9723
(у HEX: 9BC247DF529EC419143AC8EC1EC3200D6BEC016C2B4E36651B45846F5DC1248EE0C55817FCE6F8687B23836E6D4C6B3879733BB971F13D282B9D865F817E385B)

• **$\varphi(n_1) = (p_1 - 1)(q_1 - 1)$** = 815775600676224758057541045696824762404870371950626444276565885799197011362490745978895714324986552078790370503597957020563199348303640408242604678 0987648

• **Відкрита експонента e_1** = 65537 (у HEX: 10001)

• **Секретний ключ d_1** = 153602864219366365784672116573825740697256517537737923957044463902255079118866225267857441060322169959752096861534687117715944885455039483615571993343 2833
(у HEX: 1D53F39C853E07C6811CA080753A606AA68B85A90C6E69A797FEFAF4CD3E74D775CA678F8744BB440C6135DD73DCFB61763BB71ADA3B8BDA913FBCF339331401)

Проміжні результати перевірки простого числа тестом Міллера–Рабіна (Доповнення до завдання 1)

Для простого числа *p* Абонента А програма зафіксувала такі проміжні обчислення на першій випадковій основі *x*:

- **Досліджуване число p:** `675963077232188980701358788419947927545428719191216224397110778335702130564 71`
- **Розклад $p - 1 = d \cdot 2^s$:** отримали *s* = 1 та непарне *d* = `337981538616094490350679394209973963772714359595608112198555389167851065282 35`
- **Випадкова основа x:** `207460943889466141274291606778068242145609918721164046921931616586735604337 38`
- **Перевірка НСД:** gcd(*x, p*) = 1 (спільних дільників немає, переходимо до піднесення у степінь).
- **Крок r = 0 (обчислення $x_0 = x^d \bmod p$):** отримали $x_0 = 1$.
- **Висновок:** оскільки вже на першому кроці $x^d \bmod p = 1$, число *p* є сильно псевдопростим за основою *x*. Воно успішно пройшло всі 20 раундів тесту.

Перевірка співвідношень коректності RSA (Завдання 3)

Програма автоматично перевірила всі необхідні математичні умови для згенерованих ключів:

- Прості числа не збігаються ($p \neq q$ та $p_1 \neq q_1$): **Виконується (OK)**
- Модулі обчислено правильно ($n = p \cdot q$ та $n_1 = p_1 \cdot q_1$): **Виконується (OK)**
- Умова протоколу розсилання ключів ($p \cdot q \leq p_1 \cdot q_1$, тобто $n \leq n_1$): **Виконується (OK)**
- Взаємна простота експоненти з функцією Ойлера (gcd(*e, φ(n)*) = 1 та $\gcd(e_1, \varphi(n_1)) = 1$): **Виконується (OK)**
- Оберненість секретного ключа ($(e \cdot d) \bmod \varphi(n) = 1$ та $(e_1 \cdot d_1) \bmod \varphi(n_1) = 1$): **Виконується (OK)**

Чисельні значення прикладів відкритого тексту, шифротексту та цифрового підпису (Завдання 4)

Для Абонента А (шифрування від А до В та цифровий підпис А):

• Відкрите повідомлення M_A:

`4529442727863182305290731696215038197798370449658053427522219939594915072855751265666889560671792956104344473636707573994721462462204446016974362243469954`
(у HEX: `8A5F20B74F5FDBA4C1F409B2B2D6BBC2EB2D4BFF5D6884678BAAACD12A62247803DAB0AB44A2B8C388B1D4FAEA4C4FEF1EC5816F3C560CD9400E312CBBFC282`)

• **Криптограма $C_{A\to B}$ = Encrypt(M_A, (e_1, n_1)):**

`44729118069118060287403954912337879103019823598164761069229847643136806712150333902841465273923460775189635153738511004651131565707776706794327234853938 30`
(у HEX: `5567233260CA1DB85997C0B5D0BA8006BECC039DE53CB19231D497364E99929AAA4C913CB5A0B0479C2F5D49E617A32F5A69FDD6B479B6596BBEFDBD397783A6`)

• **Результат розшифрування абонентом B (Decrypt):** повністю збігається з початковим M_A (**True**).

• **Цифровий підпис S_A = Sign(M_A, (d, p, q)):**

`362183886272786255163626516006488651536923647502018201013366548157125768601723941531747348022175586642906297855764410469921540782192816219290998113976271 2`
(у HEX: `45272DF9AFE893CE93572FCF6BA93FD0F600106716673FBCF09D667DCE4CDEB21854629D5A0AC575C5159CEB17F7EA3D3879E58838DBE8115BD9ED05F1BE7E18`)

• **Перевірка підпису Verify(M_A, S_A, (e, n)):** підпис вірний (**True**).

Для Абонента B (шифрування від B до A та цифровий підпис B):

• **Відкрите повідомлення M_B:**

`41530571153287057626316814230100263632522270118531851977455741567946039856097344736615346617177804375745317994763525471101208744625461730386370219374938 47`
(у HEX: `4F4BB867075A264BB516B404C1276C49D8CD18FF5D47CAEAD6572473B6EA4BC3FB75ABEE081ADB088052C10219EFCC3776B2D48F74B0622281DDC3AA3D976757`)

• **Криптограма $C_{B\to A}$ = Encrypt(M_B, (e, n)):**

`89201480186903468990223106339770728425203021022557301421072861721569896552462065084598488415090969264396193586602655694324006510986906919073761127434365 9`
(у HEX: `110813D8E0428EEAE8D8D354DD98BEAA8B134DFFC26E76E47C561234A5FFDF67E9C17B20F611D94CC7D1B34F0535C1EAE714EE23AC50670A1979DFC8156454EB`)

• **Результат розшифрування абонентом A (Decrypt):** повністю збігається з початковим M_B (**True**).

• **Цифровий підпис S_B = Sign(M_B, (d_1, p_1, q_1)):**

`55094774593197165994154100483843872040584806707503968883732212074037382857488654946890944295040482936204201295699308609558353507730270989377100641065037 40`

• **Перевірка підпису Verify(M_B, S_B, (e_1, n_1)):** підпис вірний (**True**).

Протокол конфіденційного розсилання ключів (Завдання 5)

Абонент A передає абоненту B секретний ключ k ($0 < k < n$) по відкритому каналу за допомогою процедур **SendKey()** та **ReceiveKey()**:

1. Вибір секретного ключа k абонентом A:

k = 57126444834000538459355973557533258414309064699797210098853161638911948850299598394896541956475823179923550234711593817859094266741793521093308289303525 03

2. Створення підпису відправником A (процедура SendKey):

Абонент A обчислює підпис $S = k^d \bmod n$ за допомогою **свого секретного ключа d** (це забезпечує **підтвердження справжності відправника**):

S = 17307958925325902847526041752943603791563320466148133706488262426586954330431636797260799627110869054660034639264493000583779390425322694014243614641265 63

3. Шифрування пакета для отримувача B (процедура SendKey):

Абонент A шифрує k та S за допомогою **відкритого ключа отримувача B (e_1, n_1)** (це забезпечує **конфіденційність** у відкритому каналі):

$k_1 = k^{e_1} \bmod n_1$ = 29899908576587314492605821402856113238246439085275863607351587627977151470174236086802071407471941195050972375159104795643783176475722054998234091963528 39

$S_1 = S^{e_1} \bmod n_1$ = 66291354663271272836229107608493816954731236938277935071561940990256193501681031603348360160769236166853099051783789841158349104436809152119055835758667 16

4. Розшифрування пакета отримувачем B (процедура ReceiveKey):

Абонент B за допомогою **свого секретного ключа d_1** обчислює $k = {k_1}^{d_1} \bmod n_1$ та $S = {S_1}^{d_1} \bmod n_1$ і повністю відновлює початкові числа k та S.

5. Перевірка підпису відправника A (процедура ReceiveKey):

Абонент B за допомогою **відкритого ключа відправника A (e, n)** перевіряє рівність $k == S^e \bmod n$. Перевірка повертає **True**, отже ключ передано конфіденційно та його справжність підтверджено.

Перевірка взаємодії реалізації з тестовим середовищем

Для перевірки правильності роботи наших функцій ми використали тестовий вебсервіс http://asymcryptwebservice.appspot.com/?section=rsa та згенеровані HEX-значення з файлу **server_hex.txt**. На рис. 1 та рис. 2 наведено результати успішної верифікації цифрового підпису Абонента A (отримано статус **Verification: true**) та виконання операції модульного шифрування у вебсередовищі (отримано результат 3573DC7B91BDB57910B6A7ED87809C6ED5C5F3C2D8EE6081658FDC38506F2635A6FE016F7EDA390D3736A24ECB1039142F5AD552116777BC027ABC3C25D64A44).

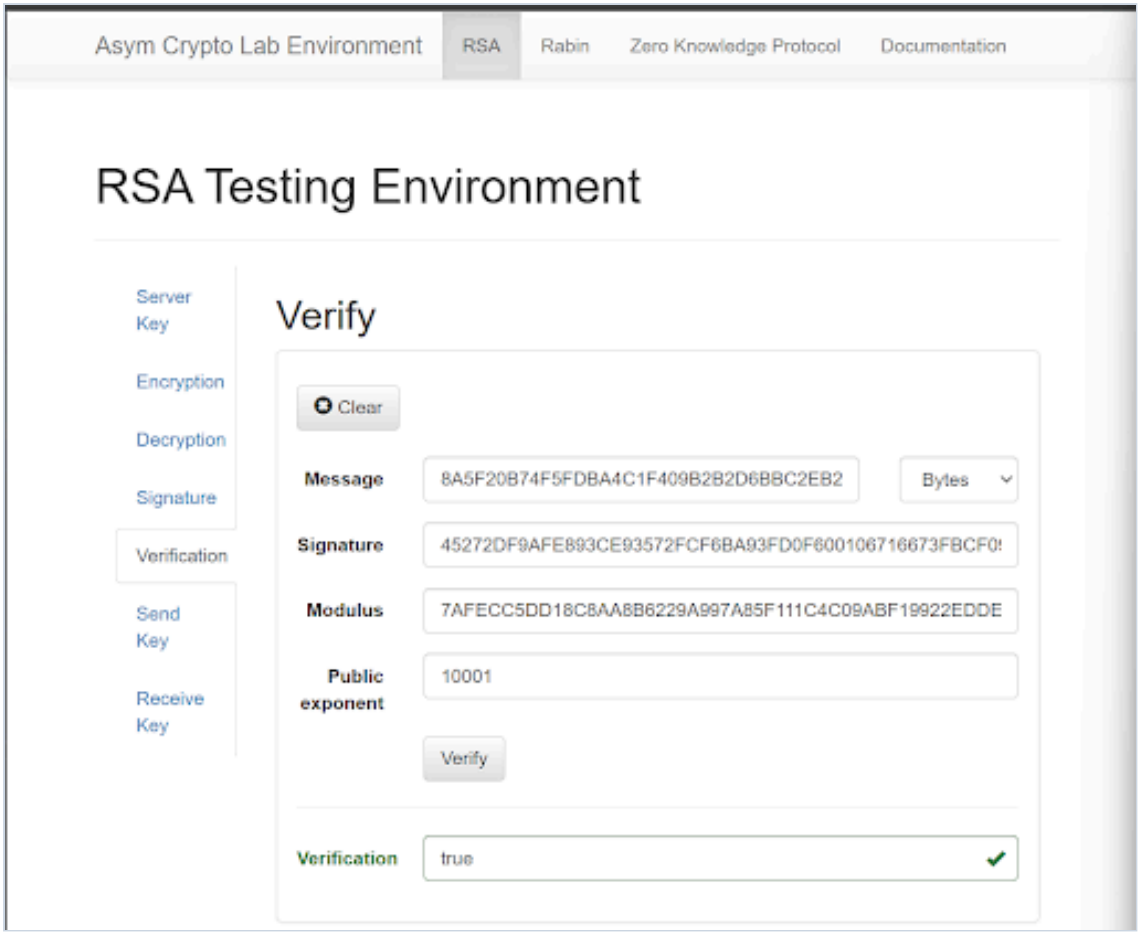


Рис. 1. Результат перевірки цифрового підпису Абонента А на тестовому сервері (Verification: true)

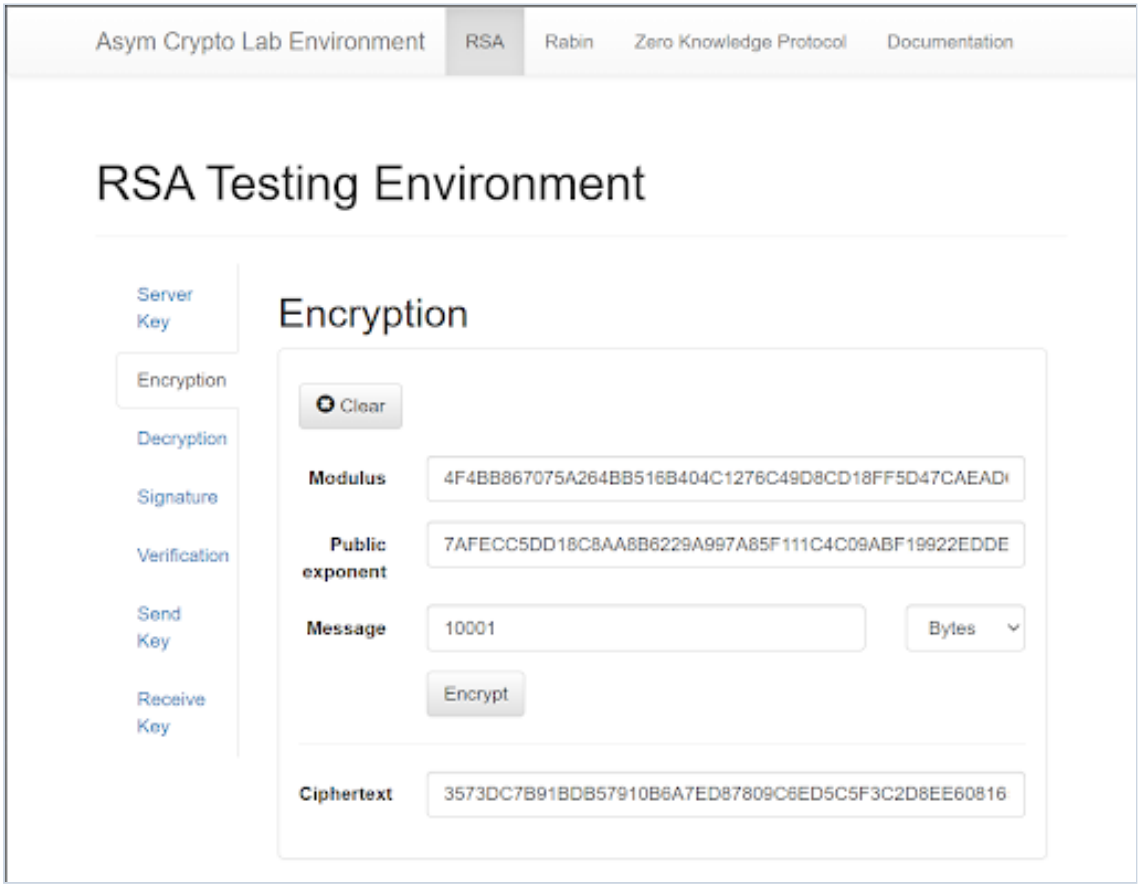


Рис. 2. Виконання операції шифрування у тестовому середовищі RSA

Контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?
Спочатку ми виставляємо старший і молодший біти числа в одиницю — це гарантує, що число буде рівно 256-бітним і непарним. Далі запускається тест пробних ділень (перевірка остачі від ділення на малі прості числа від 3 до 97), який швидко відкидає більшість складених чисел без важких обчислень. Після цього виконується 20 раундів імовірнісного тесту Міллера–Рабіна, який перевіряє сильну псевдопростоту числа з імовірністю помилки не більше 4^{-20}.

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?
Прості числа p і q мають бути різними ($p \neq q$), модуль дорівнює їх добутку ($n = p \cdot q$), для протоколу передачі ключа має виконуватись умова $n \leq n_1$, відкрита експонента e має бути взаємно простою з функцією Ойлера ($\gcd(e, \varphi(n)) = 1$), а секретна експонента d має бути оберненою до e за модулем $\varphi(n)$, тобто $(e \cdot d) \bmod \varphi(n) = 1$. Усі ці умови ми перевірили програмно у блоці перевірок (пункт 3).

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?
Для публічних операцій **Encrypt(M, (e, n))** та **Verify(M, S, (e, n))** потрібен лише відкритий ключ, а для приватних операцій **Decrypt(C, (d, p, q))** та **Sign(M, (d, p, q))** — лише секретний ключ. Усі параметри одразу не потрібні, бо в цьому і полягає суть асиметричної криптографії: відкритий ключ відомий усім, а секретний ключ знає лише його власник, тому передавати секретні параметри в публічні функції не можна.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника?
Підтвердження справжності відправника (автентифікація) забезпечується тоді, коли Абонент А підписує ключ своїм секретним ключем d ($S = k^d \bmod n$), а Абонент В перевіряє цей підпис відкритим ключем А ($k = S^e \bmod n$). Конфіденційність забезпечується тоді, коли Абонент А шифрує пакет відкритим ключем отримувача В (e_1, n_1), а Абонент В розшифровує його своїм особистим секретним ключем d_1.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім?
Після розшифрування k та S одержувач В повинен перевірити дві речі: по-перше, що ключ k входить у допустимий діапазон модуля відправника ($0 < k < n$), і по-друге, що цифровий підпис відправника А є правильним, тобто виконується рівність $S^e \bmod n == k$.

Висновок:

У ході лабораторної роботи ми реалізували мовою Python генератор 256-бітних простих чисел на базі тесту пробних ділень та імовірнісного тесту Міллера–Рабіна, побудували криптосистему RSA з 512-бітними модулями для двох абонентів та написали 7 обов'язкових процедур (**GenerateKeyPair**, **Encrypt**, **Decrypt**, **Sign**, **Verify**, **SendKey**, **ReceiveKey**). На практиці перевірили шифрування, створення електронного підпису та протокол конфіденційного розсилання ключів по відкритому каналу, а також підтвердили правильність обчислень за допомогою тестового вебсервера.